Monday
1

Comparision of Algorithms and/or Data Structures

Asymptotic Analysis
Amortized

2017

Counting Instructions to solve a problem
or count of instructions to be executed
to solve a problem.

Tuesday
2

e.g. (I)

```
def print4Hellos():
    print("Hello")
    print("Hello")
    print("Hello")
    print("Hello")
```

(I)

time taken by (I) $\propto$ 4

Wednesday
3

(II)

```
def print100Hellos():
    i=1
    while i<=100:
        print("Hello")
        i=i+1
```

(II)

time taken by (II) $\propto$ count of instructions executed

$= 1 + 101 + 100 + 100$

$= 302$

$\approx 3 \times 100$

Thursday
4

2017

Usually time taken should be a function of ~~data~~ size / input size, for which is generally a variable n is used so, mostly running time is denoted by T(n).

Friday
5

(III) def printNHellos(~~int~~ n): (III)

```
1    i = 1
2    while i <= n:
3        print("Hello")
4        i = i + 1
```

instruction (I) executed only once

(II) executed n+1 times

Saturday
6

Sunday
1

(II) time evaluated as true and last time as false

(III) executed n times

(iv) exectuted n time

so $T(n) = 1 + (n+1) + n + n$

$= 3n + 2$

Monday
1

In the way/procedure we used for last example, the running time of any algorithm can be measured as a function of its input size/data size.

Tuesday
2

Comparison of functions

which is of the following is efficient

$2n^2+3n+1$ or $3n^2+10n+2$

we only interested for larger n as for, smaller data/input size, algorithm usually take small time to execute.

Wednesday
3

for larger n, the above two function can be approximated as

$2n^2$ $3n^2$

by ignoring lessor significant term when n is very large.

Thursday 4 — 2017

there two functions have same trend/ rate of growth, despite they differs by a constant factor 2/3.

we are more interested in rate of growths of functions

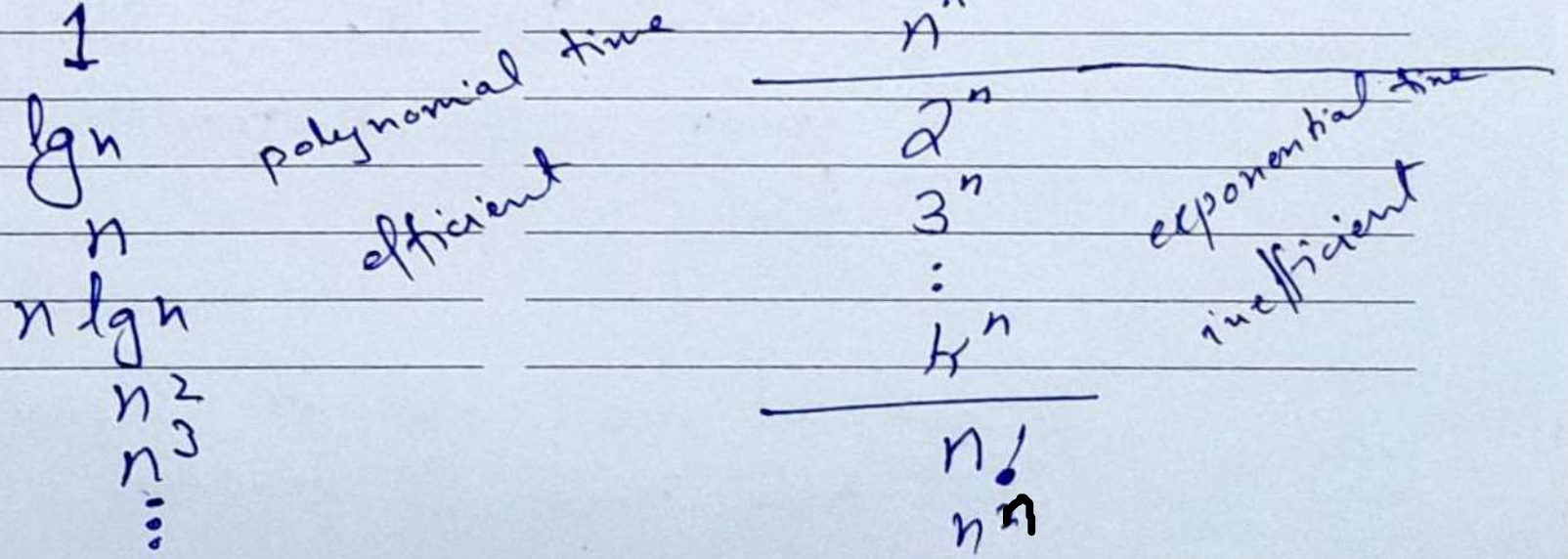
Friday 5

so $2n^2$ and $3n^2$ have same growth rate and we can say that they are asymptotically same order of n^2.

We normally classify algorithms into following

Saturday 6 — Sunday 7

classes



Monday 1

Θ : a function $f(n)$ belongs to $\Theta(g(n))$, if they have same rate of growth.

2017

Tuesday 2

O : a function $f(n)$ belongs to to $O(g(n))$, if $g(n)$ has equal or higher rate of growth than $f(n)$

Wednesday 3

Ω : a function $f(n)$ belongs to to $\Omega(g(n))$, if $g(n)$ has equal or lower rate of growth than $f(n)$

O and ω are used for higher and lower only rate of growths

Thursday
4

if $T(n) = 3n^3 + 4n^2 + 5n + 7$

2017

then

$T(n) \in \Theta(n^3)$

$\in O(n^3), O(n^4), O(2^n)$

$O(n!)$ etc

Friday
5

$\notin O(n^2), O(\lg n), O(1)$

etc

$\in \Omega(n^3), \Omega(n^2), \Omega(n \lg n)$

$\Omega(\lg n), \Omega(1)$, etc

Saturday
6

Sunday
7

$\notin \Omega(n^4), \Omega(2^n)$, etc

Here

1) if $T(n) \in O(f(n))$ and $\Omega(f(n))$

$\Rightarrow T(n) \in \Theta(f(n))$

2) if $T(n) \in O(f(n))$

$\Rightarrow f(n) \in \Omega(T(n))$ and vice versa

Monday
1

2017

if $T(n) = f(n) + g(n)$

$= \Theta(\max(f(n), g(n)))$

if $T(n) = f(n) \cdot g(n)$

$= \Theta(f(n) \cdot g(n))$

if $T(n)$ is polynomial of degree k, then

Tuesday
2

$T(n) = \Theta(n^k)$

if $T(n) = \log^* n = O(n)$

Wednesday
3

if loop variable ↑ increment / decrement / Advanced by addition/subtraction of a constant, the loop generally provide $O(n)$ iterations.

if loop variable advanced by multiplication/division by a constant, the loop generally provide $O(\lg n)$ iterations.

Thursday
4

Analysis of Iterative Algorithms

2017

i) for Consective structures, running times should be added

ii) for if, ifelse, case, structure, running time of maximum of them be taken

Friday
5

iii) for loops,

the running time of body should be multiplied by number of iterations of the loop

iv) for nested loops

the evaluation should be inside out.

Saturday
6

Sunday
7

v) for functions

the evaluation should be inside out.

*) we can have good estimate for loops using summations (Σ)

eg

a simple example is presented earlier.

Monday
1

2017

Analysis of Recursive Algorithm

The running time of algorithm should be sum of

all individual recursive calls w.r.t there data size / input size in comparison to actual call.

Tuesday
2

and

time taken by rest of code of algorithm.

this gives us, recurrance relation.

Base case is considered seperately.

later, the recurrence be solved.

Wednesday
3

e.g

```
def RAlgo(n):                          T(n)
    if n == 1:
        return 1
    else
        x = RAlgo(n-1)                 T(n-1)
        y = RAlgo(n/2)                 T(n/2)
        for j in range(n)
            print(j+x-y)
        return x+y
```

rest of code has running time $\Theta(n)$

$$T(n) = T(n-1) + T(\tfrac{n}{2}) + \Theta(n)$$

$$T(1) = \Theta(1)$$

Thursday
4

ex1

```
1  def ex1(n):                    T(n)
2      i = 1
3      while i < n:
4          j = 1
5          while j < i:
6              print (i+j)
7              j = j+1
8          i = i+1
```

2017

Friday
5

Analysis

for inner loop of lines 4 → 6, the body of loop is $\Theta(1)$, while loop iterates i times so, its running time is $\Theta(i)$

Saturday
6

Sunday
7

for loop of lines 2 to 7, the body of loop is $\Theta(i)$, which at maximum be $\Theta(n)$ and loop iterates n times, so running time of outer loop is

$$T(n) = O(n \cdot n) = O(n^2)$$

o not theta

Monday
1

ex 2

```
def ex2(n):                    # T(n)
    i = 1
    while i <= n:
        j = 1
        while j <= i:
            print(i)
            j = j + 1
        i = i * 2
```

Tuesday
2

On the similar grounds as previous example we can say that, as outer loop has $\lg n$ iterations, so the running time of algo should be

$$T(n) = O(n \lg n)$$

Wednesday
3

Let it treat other way. for better results

$$T(n) = 1 + 2 + 4 + 8 + 16 + \cdots n$$

$$= 1 + 2 + 2^2 + 2^3 + 2^4 + \cdots 2^{\log_2 n}$$

$$= \frac{2^{\log_2 n + 1} - 1}{2 - 1} = \frac{2 \cdot 2^{\lg n} - 1}{1} = 2n - 1$$

$$= \Theta(n)$$

Thursday 4

ex 3

```
def sum(n):                    # T(n)
    if n==1:
        return 1
    s = sum(n-1)               # T(n-1)
    return s+n
```

2017

Friday 5

$T(n) = T(n-1) + \theta(1)$

$T(1) = \theta(1)$ $\Rightarrow$ $T(n-1)+1$ —(I)

1 —(II)

replace n by (n-1) in (I)

$T(n-1) = T((n-1)-1) + 1$

$= T(n-1-1)+1$

$= T(n-2)+1$

Saturday 6

put in (I)

Sunday 7

$T(n) = T(n-2)+1+1$

$= T(n-2)+2$ —(III)

replace n by (n-2) in (I)

$T(n-2) = T(n-2-1)+1$

$= T(n-3)+1$

put in (III)

$T(n) = T(n-3)+1+2$

$= T(n-3)+3$ —(IV)

Monday
1

2017

replace n by $n-3$ in (1)

$$T(n-3) = T(n-3-1) + 1$$
$$= T(n-4) + 1$$

put in (iv)

$$T(n) = T(n-4) + 1 + 3$$

Tuesday
2

$$= T(n-4) + 4 \quad \text{——— (v)}$$

generalising (i), (iii) (iv), (v), ...

$$T(n) = T(n-k) + k \quad \text{——— (vi)}$$

for base case $n-k = 1$

Wednesday
3

$$\Rightarrow n-1 = 1$$
$$\Rightarrow k = n-1$$

put in (vi)

$$T(n) = T(1) + n-1$$
$$= 1 + n - 1$$
$$= n$$

$$\Rightarrow T(n) = \Theta(n)$$

Thursday
4

2017

ex4

```
def power(b, n):                    T(n)
    if n == 1:
        return b
    t1 = power(b, int(n/2))         T(n/2)
    t2 = power(b, n-int(n/2))       T(n/2)
    return t1 * t2
```

Friday
5

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 \quad \text{——— (I)}$$

$$T(1) = 1 \quad \text{——— (II)}$$

replace n by $n/2$ in eq (I)

$$T(n/2) = 2T\left(\frac{n/2}{2}\right) + 1$$

Saturday
6

$$= 2T\left(\frac{n}{2^2}\right) + 1$$ Sunday

put in (I)

$$T(n) = 2\left[2T\left(\frac{n}{2^2}\right) + 1\right] + 1$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2 + 1 \quad \text{——— (III)}$$

replace n by $n/2^2$ in eq (I)

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n/2^2}{2}\right) + 1 = 2T\left(\frac{n}{2^3}\right) + 1$$

Monday
1

2017

put in (III)

$$T(n) = 2^2\left[2T\left(\frac{n}{2^3}\right)+1\right]+2+1$$

$$= 2^3 T\left(\frac{n}{2^3}\right)+2^2+2+1 \quad \text{——(IV)}$$

replace n by $n/2^3$ in eq (I)

Tuesday
2

$\equiv$

$$T(n) = 2^4 T\left(\frac{n}{2^4}\right)+2^3+2^2+2+1 \quad \text{——(V)}$$

generalising (I), (III), (IV), (V), etc

$$T(n) = 2^k T\left(\frac{n}{2^k}\right)+2^{k-1}+\cdots+2^3+2^2+2+1$$

——(VI)

Wednesday
3

for base case $\frac{n}{2^k}=1$

$$\Rightarrow 2^k = n$$

$$\Rightarrow k = \lg n$$

put in (VI)

$$T(n) = 2^{\lg n} T(1) + 2^{\lg n-1} + \cdots + 2^3+2^2+2^1+2^0$$

Thursday
4

2017

$$= 1 + 2 + 2^2 + 2^3 + \cdots 2^{\lg n}$$

$$= \frac{2^{\lg n + 1} - 1}{2 - 1}$$

$$= 2 \cdot 2^{\lg n} - 1$$

Friday
5

$$= 2 \cdot n - 1$$

$$= \Theta(n)$$

ex 5

```
def fib(n):                                   T(n)
    if n==1 or n==2:
        return n                              T(n-1)
    return fib(n-1) + fib(n-2)                T(n-2)
```

Saturday
6

Sunday

so

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

$$\leq 2T(n-1) + 1 \quad \text{——— (I)}$$

$$T(1) = T(2) = \Theta(1) = 1 \quad \text{——— (II)}$$

Monday
1

2017

replace n by n-1 in eq (1)

$$T(n-1) \leq 2T(n-1-1) + 1$$

$$= 2T(n-2) + 1$$

put in (1)

$$T(n) \leq 2[2T(n-2)+1] + 1$$

Tuesday
2

$$= 2^2 T(n-2) + 2 + 1$$

$$\vdots$$

$$T(n) \leq 2^3 T(n-3) + 2^2 + 2 + 1$$

$$\vdots$$

$$T(n) \leq 2^4 (n-4) + 2^3 + 2^2 + 2 + 1$$

Wednesday
3

$$\vdots \quad \leq 2^k (n-k) + 2^{k+1} \cdots + 2^3 + 2^2 + 2 + 1$$

for base case

$$n-k = 1$$

$$\Rightarrow k = n-1$$

so $$T(n) \leq 2^{n-1} T(1) + 2^{n-2} + 2^3 + 2^2 + 2 + 1$$

Thursday 4

$$= 2^{n-1} + 2^{n-2} + \cdots 2^3 + 2^2 + 2 + 1$$ 2017

$$= \frac{2^{n-1+1} - 1}{2 - 1}$$

$$= 2^n$$

Friday 5 so $T(n) = O(2^n)$ it is not $\Theta(2^n)$ here

---

ex 6 Let we have

$$T(n) = 3T\left(\frac{n}{2}\right) + n \quad \text{---(I)}$$

$$T(1) = 1 \quad \text{---(II)}$$

Saturday 6 replace n by $\frac{n}{2}$ in equation (I) Sunday 7

$$T\left(\frac{n}{2}\right) = 3T\left(\frac{n/2}{2}\right) + \frac{n}{2}$$

$$= 3T\left(\frac{n}{2^2}\right) + \frac{n}{2}$$

put in (I)

$$T(n) = 3\left[3T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right] + n$$

$$= 3^2 T\left(\frac{n}{2^2}\right) + \frac{3}{2}n + n \quad \text{---(III)}$$

Monday
1

2017

replace n by $n/2^2$ in eq (1)

$$T\left(\frac{n}{2^2}\right) = 3T\left(\frac{n/2^2}{2}\right) + \frac{n}{2^2}$$

$$= 3T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \quad \text{— (iv)}$$

put in 3

Tuesday
2

$$T(n) = 3^2\left[3T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right] + \frac{3n}{2} + n$$

$$= 3^3 T\left(\frac{n}{2^3}\right) + \left(\frac{3}{2}\right)^2 n + \frac{3n}{2} + n \quad \text{— (v)}$$

$$\vdots$$

$$T(n) = 3^4 T\left(\frac{n}{2^4}\right) + \left(\frac{3}{2}\right)^3 n + \left(\frac{3}{2}\right)^2 n + \left(\frac{3}{2}\right) n + n$$

— (vi)

Wednesday
3

generalizing

$$T(n) = 3^k T\left(\frac{n}{2^k}\right) + \left(\frac{3}{2}\right)^{k-1} n + \cdots + \left(\frac{3}{2}\right)^2 n + \left(\frac{3}{2}\right) n + n$$

for base case

$$\frac{n}{2^k} = 1$$

$$\Rightarrow 2^k = n \Rightarrow k = \lg n$$

Thursday
4

$$T(n) = 3^{\lg n} T(1) + \left(\frac{3}{2}\right)^{\log n-1} + \cdots + \left(\frac{3}{2}\right)^{n} + \left(\frac{3}{2}\right)^{n+1}$$

$$= n^{\lg 3} + \left[1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \cdots \left(\frac{3}{2}\right)^{\lg n-1}\right] n$$

$$= n^{\lg 3} + \frac{\left(\frac{3}{2}\right)^{\lg n-1+1} - 1}{\frac{3}{2} - 1} n$$

Friday
5

$$= n^{\lg 3} + \frac{\left(\frac{3}{2}\right)^{\lg n} - 1}{1/3} n$$

$$= n^{\lg 3} + 3n\left(n^{\lg \frac{3}{2}} - 1\right)$$

Saturday
6

Sunday
7

$$= n^{\lg 3} + 3n\left(n^{\lg 3 - \lg 2} - 1\right)$$

$$= n^{\lg 3} + 3n \cdot n^{\lg 3 - 1} - 3n$$

$$= n^{\lg 3} + 3 n^{\lg 3} - 3n$$

$$= 4 n^{\lg 3} - 3n$$

$$= \Theta(n^{\lg 3}) = \Theta(n^{1.59})$$

Monday
1

2017

ex7 def ex7(n):

```
1    i = 1
2    while i <= 2**n:
3        j = 1
4        while j <= n
5            print(i+j)
6            j = j*3
7        i = i*2
```

Tuesday
2

for loop of lines 4 to 6

Wednesday
3

$j \Rightarrow 1, 2, 4, 8, 16 \ldots\ldots n$

$2^0\ 3^1\ 3^2\ 3^3\ 3^4 \qquad\qquad 3^{\lg n}$

$1+1+1+1+1+\ldots\ldots+1$

$= \lg_3 n + 1$

for loop of lines 2 to 7

Thursday
4

Friday
5

Saturday
6

Sunday
7

2017

$i \Rightarrow$ 1 2 4 8 16 ... 2^k

2^0 2^1 2^2 2^3 2^4 ... 2^k

$= (\lg_3 n+1) + (\lg_3 n+1) + (\lg_3 n+1) + \cdots\cdots + (\lg_3 n+1)$

$= (n+1)(\lg_3 n+1)$

$= n\lg_3 n + n + \lg_3 n + 1$

$= \Theta(n\lg_3 n) = \Theta(n \lg n)$

---

what if, (a) line 6 is $j = j+3$

(b) line 7 is $i = i+1$

(c) line 7 vs $i = i+2$

(d) line 6 is $j = j+1$
and 7 is $i = i+1$

Monday
1

WORST, BEST and AVERAGE CASES (EVERY CASE)

2017

ex 1

```
def search(A, n, x):                    T(n)
1       i = 0
2       while i <= n-1:
3           if A[i] == x:
4               return i
5       raise Exception("Not found")
```

Tuesday
2

Cases 1: x is not in A anywhere
line 3 evaluates to false n times

$\Rightarrow T(n) = \Theta(n)$

2: x is index 0 of A
line 3 evaluates to true in 1st iteration and line 4 stops iteration of loop just in its 1st iteration.

Wednesday
3

Thursday 4

2017

$\Rightarrow T(n) = \Theta(1)$

3: if x is somewhere in array A the $T(n)$ varies from 1 to n.

Friday 5

Here we can discuss three cases.

1) WORST CASE

when loop iterates maximum times it can.

Saturday 6

2) BEST CASE

Sunday 7

when loop iterates minimum times it can.

3) AVERAGE CASE

How many times loop iterates on the average. It may required statistical/ probabilistic analysis.

(not always) ⇒ Usually it is similar to WORST CASE.

Monday
1

2017

ex2

```
1  def avg(A, n):
2      s = 0
2      i = 0
3      while i < n:
4          s = s + A[i]
5          i = i + 1
6      return s
```

Tuesday
2

The loop always iterates n times, so in above example, no WORST, BEST & AVERAGE case analysis. And we have what we can say EVERY CASE

$$T(n) = \theta(n)$$

Wednesday
3

Thursday 4

Space Requirments

2017

Estimation of the memory (temporary or auxillary) required to solve a solve through some algorithm.

Friday 5

eg

```
def fib1(n):
    f1 = f2 = 0
    i = 1
    while i <= n:
        t = f1
        f1 = f2
        f2 = t + f1
        i = i + 1
    return f2
```

Saturday 6

requires 3 variables, so
$M(n) = O(1)$

```
def fib2(n):
    t = [0] * n
    t[1] = 1
    i = 2
    while i <= n:
        t[i] = t[i-1] + t[i-2]
        i = i + 1
    return t[n]
```

Sunday 7

requires an array and a variable, so
$M(n) = n + 1 = O(n)$

```
def sum(A, n):
    s = i = 0
    while i < n:
        s += A[i]
        i = i + 1
    return s
```

requires s, i & n as extra memory, A has data to pre
so $M(n) = O(1)$